

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ

FACULTAD DE CIENCIAS E INGENIERÍA

INTELIGENCIA ARTIFICIAL (1INF24)

Examen 1 (Parte 1) – Semestre 2025-1

Indicaciones generales:

- Duración máxima: **45 minutos**.
- Para responder las preguntas de esta parte **no está permitido ningún material**
- **Solo se permite Editor de Texto** para responder las preguntas
- Cualquier indicio de plagio resultará en la anulación de la prueba.
- El archivo de respuestas nómbrelo **Examen1-Parte1_código.docx**, donde código es su código PUCP de 8 dígitos. Subirlo a Paideia antes de finalizar
- Se tomará en cuenta en la calificación la precisión de la respuesta y el uso adecuado del lenguaje.

Pregunta 1 (2 puntos)

Se tiene dos autos, A y B, en un estacionamiento de 4 posiciones, tal como mostrado en la figura. Se quiere mover los carros para que queden en el estado objetivo mostrado. Solo se puede mover un carro a la vez a una posición desocupada. No se puede mover en diagonal.

Estado Inicial

A	
B	
	B
	A

Estado Objetivo

Indique el valor de **d**, **b** y **m** del árbol de búsqueda desarrollado. ¿Cuál sería el método de búsqueda sin información que usted recomendaría para resolver dicho problema? Justifique en cuanto a costo temporal, espacial y optimalidad.

Respuestas:

 1. Análisis del problema

Tenemos 4 posiciones (2×2) y 2 autos (A y B) que deben intercambiar sus lugares.

Estado inicial:

CSS

 Copy code

A B

--

Estado objetivo:

CSS

 Copy code

B A

--

(El subrayado representa posiciones vacías o desocupadas)

- Solo se puede mover **un carro a la vez** hacia una **posición vacía adyacente (no diagonal)**.
- En cada paso, se genera un nuevo estado moviendo A o B.

b (factor de ramificación):

Número promedio de hijos por nodo.

→ En este problema, cada estado puede generar **2 movimientos posibles** (mover A o mover B), entonces

b = 2.

d (profundidad de la solución):

Es la cantidad mínima de movimientos para alcanzar el objetivo desde el inicial.

m (profundidad máxima del árbol):

Es la profundidad límite de búsqueda, considerando todas las posibles permutaciones.

→ En un espacio tan pequeño (4 posiciones, 2 ocupadas), el máximo sería el número de configuraciones alcanzables, que ronda $m \approx 4$.

✓ **m = 4.**

✓ **Recomendado: Búsqueda en Amplitud (BFS)**

Justificación:

- **Complejidad:** Sí (encuentra la solución si existe).
- **Optimalidad:** Sí, porque todos los pasos tienen igual costo.
- **Complejidad temporal:** $O(b^d) = 2^3 = 8 \rightarrow$ razonable.
- **Complejidad espacial:** $O(b^d) = 8 \rightarrow$ manejable por el tamaño reducido.

💡 **Conclusión:**

BFS garantiza el camino más corto (3 movimientos) y, dado el tamaño pequeño del espacio de estados, es óptima tanto en tiempo como en espacio.

✗ **Alternativa menos recomendable: DFS**

- **Ventaja:** usa poca memoria ($O(b \cdot m)$).
- **Desventaja:** puede seguir un camino largo o cíclico y no garantiza la mejor solución (no óptima).
→ Solo sería conveniente si el árbol fuera muy grande y la memoria limitada, lo cual no aplica aquí.

◆ 1. ¿Qué representan b, d y m?

Símbolo	Significado	Qué mide	
b	<i>Factor de ramificación</i>	Cuántos hijos genera cada nodo en promedio (cuántos movimientos posibles por estado).	
d	<i>Profundidad de la solución</i>	Cuántos movimientos mínimos se necesitan para llegar al objetivo desde el estado inicial.	
m	<i>Profundidad máxima del árbol</i>	Cuán profundo podría llegar la búsqueda (en el peor caso).	

Pregunta 2 (2 puntos)

Defina brevemente qué es una heurística dominante, por qué es ventajoso usar heurísticas dominantes en A* y qué haría si no hay clara dominancia entre las heurísticas que está considerando?

Respuesta – Pregunta 2

Heurística dominante:

Una heurística h_1 **domina** a otra heurística h_2 si para todo nodo n se cumple:

$$h_1(n) \geq h_2(n) \quad h_1(n) \geq h_2(n)$$

y ambas son **admisibles** (nunca sobreestiman el costo real al objetivo).

Esto significa que h_1 proporciona **estimaciones más informativas o cercanas al costo real** que h_2 .

Ventajas de usar heurísticas dominantes en A*

- Si h_1 domina a h_2 , A* con h_1 **nunca expande más nodos** que con h_2 .
- Por tanto, es **más eficiente**: explora menos caminos y llega antes a la solución óptima.
- Mantiene la **admisibilidad y optimalidad** del algoritmo (siempre que h_1 también sea admisible).



En resumen:

→ **Más fuerte = menos búsqueda = mismo resultado óptimo.**



Si no hay clara dominancia entre las heurísticas

Cuando ninguna heurística domina a las otras, se puede combinar de forma segura:

$$h(n) = \max(h_1(n), h_2(n), \dots, h_k(n)) \quad h(n) = \max(h_1(n), h_2(n), \dots, h_k(n))$$

Esta combinación **sigue siendo admisible** y suele **producir una heurística más informativa** que cualquiera por separado, mejorando la eficiencia del algoritmo.

✓ **En resumen:**

- Heurística dominante = más informativa y eficiente.
- Se prefiere en A* porque **reduce nodos expandidos sin perder optimalidad**.
- Si no hay dominancia, usar **la combinación por máximo** entre ellas.

Pregunta 3 (2 puntos)

Explique brevemente en qué consiste el método del problema relajado para generar heurísticas y por qué las heurísticas que se generan con este método son admisibles en el problema original

🧠 **Respuesta – Pregunta 3**

Método del problema relajado:

Consiste en **simplificar o eliminar una o más restricciones del problema original**, de modo que el nuevo problema (relajado) sea **más fácil de resolver**.

Luego, el **costo de la solución óptima del problema relajado** se usa como **heurística $h(n)$** para el problema original.

💡 **Ejemplo:**

En el **8-puzzle**, si relajamos la restricción de mover solo fichas adyacentes, podemos estimar el costo contando simplemente cuántas fichas están fuera de lugar.

✓ **Por qué las heurísticas generadas así son admisibles**

Porque al quitar restricciones, el problema relajado **nunca puede costar más que el real** —es decir, su solución siempre será **igual o más barata** que la del problema original:

$$h(n) \leq h^*(n) \quad h(n) \leq h^*(n)$$

Por eso, la heurística derivada del problema relajado **nunca sobreestima** el costo verdadero y, por tanto, es **admisible**.

✓ **En resumen:**

- El problema relajado **simplifica reglas** para estimar más fácilmente.
- Su solución óptima da una **cota inferior** del costo real.
- Las heurísticas obtenidas **siempre son admisibles** en el problema original.

Pregunta 4 (2 puntos)

¿Cuál considera que es la principal ventaja de Simulated Annealing sobre Hill Climbing y cómo se controla el balance exploración/explotación en Simulated Annealing. Justifique

Respuesta – Pregunta 4

*Ventaja principal de Simulated Annealing (SA) sobre Hill Climbing (HC):

La gran ventaja es que **SA puede escapar de los óptimos locales**, mientras que **HC se queda atrapado** en ellos.

Esto ocurre porque *Simulated Annealing* **permite aceptar temporalmente soluciones peores** con cierta **probabilidad controlada por la temperatura (T)**.

→ En cambio, *Hill Climbing* solo acepta mejoras inmediatas (*greedy local search*).

 **Ejemplo:**

Si el algoritmo llega a una “colina pequeña” (óptimo local), SA puede “bajar un poco” para intentar llegar a una “montaña más alta” (óptimo global).

Balance exploración/explotación en SA

Este balance se controla mediante la **temperatura (T)**:

- **Alta T (inicio):**
 - Mayor probabilidad de aceptar soluciones peores → **más exploración** del espacio de búsqueda.
 - El algoritmo “prueba” diferentes regiones (búsqueda global).
 - **Baja T (final):**
 - Menor probabilidad de aceptar soluciones peores → **más explotación**.
 - Se concentra en refinar la mejor zona encontrada (búsqueda local).
- ♦ El proceso de **enfriamiento gradual (schedule)** permite pasar suavemente de exploración a explotación, simulando el “templado” del metal.
-

En resumen:

- **Ventaja:** *Simulated Annealing* evita el estancamiento en óptimos locales aceptando soluciones peores al inicio.
- **Control exploración/explotación:** mediante la **temperatura (T)** y su **tasa de enfriamiento**:
 - Alta T → exploración amplia.
 - Baja T → refinamiento y convergencia.
- **Justificación:** este mecanismo probabilístico equilibra la búsqueda global y local, mejorando la probabilidad de alcanzar el **óptimo global**.

Pregunta 5 (2 puntos)

¿Por qué y en qué situaciones usted preferiría usar un algoritmo ABC frente a un algoritmo PSO? Justifique con base a la teoría y propiedades de los algoritmos

 Respuesta – Pregunta 5

Preferiría usar un algoritmo ABC (Artificial Bee Colony) frente a un PSO cuando el problema tiene múltiples óptimos locales, espacios de búsqueda irregulares o naturaleza discreta o combinatoria (por ejemplo, planificación, rutas o asignación de recursos).

 Justificación teórica

Aspecto	ABC (Colonia de abejas)	PSO (Enjambre de partículas)
Naturaleza	Estocástico, combina búsqueda local y global (exploración y explotación)	Basado en movimiento continuo cooperativo (más suave y determinista)
Diversidad de búsqueda	Alta — gracias a las abejas exploradoras (scouts) que introducen nuevas soluciones aleatorias	Menor — el enjambre tiende a converger rápidamente alrededor del mejor global (riesgo de estancamiento)
Aplicación típica	Problemas discretos o combinatorios, con muchos óptimos locales	Problemas continuos o de ajuste fino (optimización de funciones reales)
Mecanismo de escape	Las scouts reemplazan fuentes estancadas → mantiene diversidad	No tiene exploración aleatoria natural → tiende a converger prematuramente
Parámetros	Pocos y fáciles de ajustar (SN, límite, iteraciones)	Requiere calibrar inercia (ω) y coeficientes (φ_1, φ_2)

 Conclusión

Preferiría **ABC sobre PSO** cuando:

- El espacio de búsqueda es **complejo, multimodal o discreto**.
- Se requiere **mantener diversidad poblacional** y evitar convergencia prematura.
- El problema exige **equilibrar exploración y explotación** sin depender de parámetros delicados.

- ♦ En cambio, usaría **PSO** si el problema es **continuo y bien comportado**, donde la velocidad de convergencia es más importante que la exploración.
-

✓ **En resumen:**

ABC se prefiere por su mayor capacidad de **exploración y adaptación**, ideal para **problemas con muchos óptimos locales o espacios discretos**, mientras que **PSO** es más eficiente en **problemas continuos y suaves**.